Nonlinear Solvers and Implied Volatility

FRE6883 – Recitation 10

Bracket the root, follow the tangent, recover the market input

One equation, three C++ designs

$$f(x) = c$$

bisection Newton–Raphson

↓

function pointers

virtual functions

class templates

By the end

You can trace both solvers, complete their C++ implementations, compare the three ways to supply a function, and compute Black–Scholes implied volatility.

The numerical method and the representation of the function are separate design choices.

Implied volatility is an inverse problem

For known S_0, K, T, and r, Black–Scholes gives a call price

$$C = C(S_0, K, T, \sigma, r).$$

The market supplies a quote C_{quote}. We seek the volatility satisfying

$$C(S_0, K, T, \sigma, r) = C_{\text{quote}}.$$

Root-finding form

Define $f(\sigma) = C(S_0, K, T, \sigma, r)$ and target $c = C_{\text{quote}}$. Implied volatility is a solution of $f(\sigma) - c = 0$.

Start with a known test equation

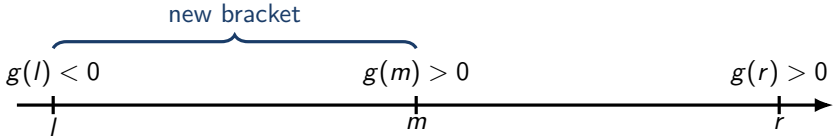
```
double F1(double x)
{
    return x * x - 2.0;
}

double DF1(double x)
{
    return 2.0 * x;
}
```

Expected positive root

Solving `F1(x) = 0` on $[0, 2]$ should return $\sqrt{2} \approx 1.4142$. This gives us a result against which to test both algorithms.

Bisection preserves a bracket



Invariant

For $g(x) = f(x) - c$, the endpoints keep opposite signs. Each step discards the half that cannot contain the sign change.

01 | Perform one bisection step – Problem

GUIDED CODE Numerical trace Compute before voting

Let

$$g(x) = x^2 - 2, \qquad [l, r] = [0, 2].$$

1. Compute $m = (l + r)/2$.
2. Evaluate the signs of $g(l)$, $g(m)$, and $g(r)$.
3. Which half-interval becomes the new bracket?

$$m = 1, \qquad g(0) = -2, \qquad g(1) = -1, \qquad g(2) = 2.$$

$g(m)$ has the same sign as $g(l)$, so the left endpoint moves to m:

$$[l, r] \leftarrow [1, 2].$$

What remains true

The new endpoint values still have opposite signs, so the continuous function still has a root inside the bracket.

02 | Validate the initial bracket – Problem

GUIDED CODE Bisection precondition — Which intervals are valid?

```
double g(double x) { return x * x - 2.0; }

// A: [-1.0, 0.0]
// B: [ 0.0, 2.0]
// C: [ 2.0, 3.0]
```

Question

For which interval do the endpoint values have opposite signs? Why should the solver reject the other two before entering its loop?

02 | Validate the initial bracket – Solution

```
double yLeft  = Fct(LEnd) - Tgt;
double yRight = Fct(REnd) - Tgt;

if (yLeft * yRight > 0.0)
    throw std::invalid_argument(
        "Bisection endpoints do not bracket a root");
```

Only B brackets the positive root

$g(0) < 0$ and $g(2) > 0$. Intervals A and C have endpoint values with the same sign.

A zero endpoint may be returned immediately.

03 | Complete the bisection update – Problem

GUIDED CODE Core loop

Preserve the sign change

```
double left = LEnd, right = REnd;
double mid = (left + right) / 2.0;
double yLeft = Fct(left) - Tgt;
double yMid  = Fct(mid) - Tgt;

if (/* yLeft and yMid have the same sign */)
{
    /* move left to mid and reuse yMid */
}
else
{
    /* move right to mid */
}
```

03 | Complete the bisection update – Solution

```
if ((yLeft > 0.0 && yMid > 0.0)
 || (yLeft < 0.0 && yMid < 0.0))
{
    left = mid;
    yLeft = yMid;
}
else
{
    right = mid;
}
```

Avoid a redundant evaluation

When `left = mid`, the already computed `yMid` becomes the new `yLeft`.

04 | Complete the bisection loop – Problem

GUIDED CODE Stopping rule

Fill update and termination

```
double left = LEnd, right = REnd;
double mid = (left + right) / 2.0;
double yLeft = Fct(left) - Tgt;
double yMid  = Fct(mid) - Tgt;

while (/* current half-width exceeds Acc */)
{
    /* preserve the bracket */
    mid = /* new midpoint */;
    yMid = Fct(mid) - Tgt;
}
return mid;
```

04 | Complete the bisection loop – Solution

```
while (mid - left > Acc)
{
    if ((yLeft > 0.0 && yMid > 0.0)
     || (yLeft < 0.0 && yMid < 0.0))
    {
        left = mid;
        yLeft = yMid;
    }
    else right = mid;

    mid = (left + right) / 2.0;
    yMid = Fct(mid) - Tgt;
}
return mid;
```

Bisection convergence is predictable

After k iterations, the original interval width $b - a$ has been reduced to

$$\frac{b-a}{2^k}.$$

To make the width no larger than ε, it is enough that

$$k \geq \log_2\left(\frac{b-a}{\varepsilon}\right).$$

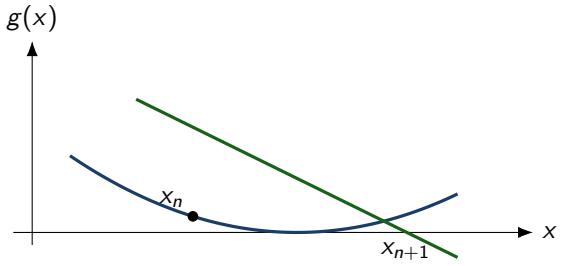
Tradeoff

Bisection is robust once a valid bracket is known, but it usually needs more iterations than Newton–Raphson near a well-behaved root.

Newton–Raphson follows the tangent

For $g(x) = f(x) - c$, the update is

$$x_{n+1} = x_n - \frac{g(x_n)}{g'(x_n)} = x_n - \frac{f(x_n) - c}{f'(x_n)}.$$



GUIDED CODE Numerical trace

Use $x_0 = 1$

For

$$f(x) = x^2 - 2, \qquad f'(x) = 2x, qquadc = 0, qquadx_0 = 1,$$

compute

$$x_1 = x_0 - \frac{f(x_0) - c}{f'(x_0)}.$$

Is x_1 above or below $\sqrt{2}$?

$$x_1 = 1 - \frac{1^2 - 2}{2(1)} = 1.5.$$

Since $1.5 > \sqrt{2}$, the first tangent step overshoots the positive root slightly.

Next step

$$x_2 = 1.5 - \frac{1.5^2 - 2}{2(1.5)} = 1.416\overline{6},$$

already close to 1.4142.

06 | Complete the Newton update – Problem

GUIDED CODE Core iteration

Fill both expressions

```
double xPrev = Guess;
double xNext =
    xPrev - /* residual at xPrev */
            / /* derivative at xPrev */;

while (/* successive guesses differ by more than Acc */)
{
    xPrev = xNext;
    xNext = /* same Newton update */;
}
return xNext;
```

06 | Complete the Newton update – Solution

```
double xPrev = Guess;
double xNext =
    xPrev - (Fct(xPrev) - Tgt) / DFct(xPrev);

while (std::fabs(xNext - xPrev) > Acc)
{
    xPrev = xNext;
    xNext =
        xPrev - (Fct(xPrev) - Tgt) / DFct(xPrev);
}
return xNext;
```

Include <cmath> for std::fabs.

07 | Diagnose Newton failure – Problem

GUIDED CODE Derivative and initial guess

What happens first?

```
double f(double x)  { return x * x * x; }
double df(double x) { return 3.0 * x * x; }

double target = 1.0;
double guess = 0.0;
double next = guess
    - (f(guess) - target) / df(guess);
```

Question

Why is the first update undefined? Which property of Newton–Raphson is missing at the chosen guess?

07 | Diagnose Newton failure – Solution

At $x_0 = 0$,

$$f'(x_0) = 3x_0^2 = 0,$$

so the Newton update divides by zero.

Practical checks

Reject or change a guess when the derivative is too close to zero. A production solver should also limit iterations and detect non-finite or out-of-range updates.

The lecture implementation assumes a sufficiently good starting guess and a nonzero derivative near the root.

Bisection or Newton–Raphson?

	Bisection	Newton–Raphson
Needs	sign-changing bracket	derivative and initial guess
Step	halve interval	follow tangent
Strength	robust bracket preservation	fast near a regular root
Failure mode	invalid bracket	small derivative or poor guess

Useful workflow

Bisection provides a dependable reference. Newton–Raphson is attractive when a reliable derivative and reasonable initial guess are available.

The solver stores numerical controls

```
class NonlinearSolver {
private:
    double Tgt;    // target c
    double LEnd;   // left bracket endpoint
    double REnd;   // right bracket endpoint
    double Acc;    // stopping tolerance
    double Guess;  // Newton initial guess
public:
    NonlinearSolver(double Tgt_, double LEnd_,
                    double REnd_, double Acc_,
                    double Guess_);
};
```

The algorithms solve `Fct(x) = Tgt`; they are not restricted to a zero target.

08 | Declare the function-pointer interface – Problem

GUIDED CODE First implementation — Complete both method signatures

```
class NonlinearSolver {
public:
    double SolveByBisect(/* pointer to f(x) */);

    double SolveByNR(
        /* pointer to f(x) */,
        /* pointer to f'(x) */);
};
```

Question

Why does bisection need one function while Newton–Raphson needs two?

08 | Declare the function-pointer interface – Solution

```
double SolveByBisect(
    double (*Fct)(double x));

double SolveByNR(
    double (*Fct)(double x),
    double (*DFct)(double x));
```

Behavior as an argument

The solver receives addresses of free functions and invokes them inside its loops without knowing their formulas.

09 | Wire the test driver – Problem

GUIDED CODE Function pointers — Complete the calls

```
NonlinearSolver solver(
    0.0, 0.0, 2.0, 0.0001, 1.0);

double byBisect = solver.SolveByBisect(/* ? */);
double byNewton = solver.SolveByNR(/* ?, ? */);

std::cout << byBisect << " " << byNewton;
```

Target

Use `F1` and `DF1` from the test equation $x^2 - 2 = 0$.

09 | Wire the test driver – Solution

```
double byBisect = solver.SolveByBisect(F1);
double byNewton = solver.SolveByNR(F1, DF1);

std::cout << std::fixed << std::setprecision(4)
          << byBisect << " " << byNewton;
// 1.4142 1.4142
```

Limitation

A plain function pointer does not carry object state. Representing $f(x, a) = x^2 - a$ requires an additional design when a varies by object.

Virtual functions let the function carry state

```
class Function {
public:
    virtual double Value(double x) = 0;
    virtual double Deriv(double x) = 0;
    virtual ~Function() {}
};

class F2 : public Function {
private:
    double a;
public:
    F2(double a_) : a(a_) {}
    double Value(double x);
    double Deriv(double x);
};
```

10 | Implement the parameterized function – Problem

GUIDED CODE Polymorphic function object — Complete both overrides

```
double F2::Value(double x)
{
    /* f(x,a) = x^2 - a */
}

double F2::Deriv(double x)
{
    /* derivative with respect to x */
}
```

Question

Which data member makes two `F2` objects represent different functions?

10 | Implement the parameterized function – Solution

```
double F2::Value(double x)
{
    return x * x - a;
}

double F2::Deriv(double x)
{
    return 2.0 * x;
}
```

Stateful behavior

`F2 f10(10.0)` represents $x^2 - 10$; `F2 f25(25.0)` represents $x^2 - 25$ using the same class implementation.

11 | Replace function calls by virtual dispatch – Problem

GUIDED CODE Second implementation

Complete the two expressions

```
double NonlinearSolver::SolveByNR(Function* Fct)
{
    double xPrev = Guess;
    double xNext = xPrev
        - (/* function value */ - Tgt)
          / /* derivative value */;
    // ... repeat until accurate ...
    return xNext;
}
```

11 | Replace function calls by virtual dispatch – Solution

```
double xNext = xPrev
    - (Fct->Value(xPrev) - Tgt)
      / Fct->Deriv(xPrev);
```

Run-time selection

The static type is `Function*`. The dynamic type selects `F1::Value` or `F2::Value`, together with the corresponding derivative.

A class template removes the base-class contract

```
template <typename Function>
class NonlinearSolver {
private:
    double Tgt, LEnd, REnd, Acc, Guess;
public:
    double SolveByBisect(Function* Fct);
    double SolveByNR(Function* Fct);
};
```

Compile-time requirement

The supplied type must provide `Value(double)` and `Deriv(double)`. It does not need to inherit from `Function`.

12 | Instantiate the template solver – Problem

GUIDED CODE Third implementation

Complete the two solver types

```
F1 myF1;
F2 myF2(10.0);

NonlinearSolver</* ? */> solver1(
    0.0, 0.0, 2.0, 0.0001, 1.0);
NonlinearSolver</* ? */> solver2(
    0.0, 0.0, 5.0, 0.0001, 3.0);

std::cout << solver1.SolveByNR(&myF1) << "\n";
std::cout << solver2.SolveByNR(&myF2) << "\n";
```

12 | Instantiate the template solver – Solution

```
NonlinearSolver<F1> solver1(
    0.0, 0.0, 2.0, 0.0001, 1.0);
NonlinearSolver<F2> solver2(
    0.0, 0.0, 5.0, 0.0001, 3.0);

std::cout << std::fixed << std::setprecision(4);
std::cout << solver1.SolveByNR(&myF1) << "\n";
std::cout << solver2.SolveByNR(&myF2) << "\n";
// 1.4142
// 3.1623
```

What changed?

The algorithm is unchanged. The compiler creates a solver specialization for each concrete function type.

Three implementations of the same algorithm

Design	Function supplied as	Selection	Carries state?
Function pointer	free-function address	compile time	awkward
Virtual interface	`Function*`	run time	yes
Class template	concrete type	compile time	yes

One solver idea

All three versions perform the same bisection or Newton updates. Their difference is how the solver obtains *value* and *derivative*.

13 | Multiple inheritance – What happens?

GUIDED CODE Read the hierarchy

Predict the compiler response

```
class Student {
public:
    void SetId(int id);
};
class Employee {
public:
    void SetId(int id);
};
class GradAssistant : public Student,
                      public Employee {};

GradAssistant ga;
ga.SetId(17);       // OK, warning, or error?
```

13 | Multiple inheritance – What happens?

GUIDED CODE Read the hierarchy — Predict the compiler response

```
class Student {
public:
    void SetId(int id);
};
class Employee {
public:
    void SetId(int id);
};
class GradAssistant : public Student,
                      public Employee {};

GradAssistant ga;
ga.SetId(17);       // OK, warning, or error?
```

Answer

Compilation error: `SetId` is ambiguous. Both base classes provide a candidate.

Resolve ambiguity; understand virtual inheritance

```
ga.Student::SetId(17);   // explicitly choose a base

class Student : public virtual Person { /* ... */ };
class Employee : public virtual Person { /* ... */ };
```

Why `virtual Person`?

If both paths derive from `Person`, ordinary multiple inheritance creates two `Person` subobjects. Virtual inheritance makes the final object share one `Person` base.

Theme 08 connection

The implied-volatility adapter inherits pricing behavior from `EurCall` and the solver interface from `Function`. The two bases have different roles, so there is no shared-base diamond there.

Black–Scholes supplies both ingredients

$$C(\sigma) = S_0 N(d_+) - K e^{-rT} N(d_-), \qquad d_\pm = \frac{\log(S_0/K) + (r \pm \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}.$$

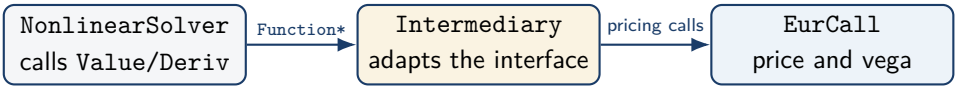
Newton–Raphson also needs

$$C'(\sigma) = \text{Vega} = S_0\sqrt{T}\,\phi(d_+).$$

Financial meaning

For a standard European call with $T > 0$, vega is positive. The Black–Scholes call price therefore increases with volatility, which supports a unique implied volatility inside a valid bracket.

Adapter between finance code and solver code



Separation of concerns

The solver knows no option formula. The option class knows no root-finding algorithm. The intermediary translates one interface into the other.

14 | Complete the implied-volatility adapter – Problem

GUIDED CODE Multiple inheritance

Translate the solver interface

```
class Intermediary : public EurCall,
                     public Function {
private:
    double S0, r;
public:
    Intermediary(double S0_, double r_,
                 double T_, double K_)
      : EurCall(T_, K_), S0(S0_), r(r_) {}

    double Value(double sigma) {
        return /* ? */;
    }
    double Deriv(double sigma) {
        return /* ? */;
    }
};
```

14 | Complete the implied-volatility adapter – Solution

```
double Value(double sigma)
{
    return PriceByBSFormula(S0, sigma, r);
}

double Deriv(double sigma)
{
    return VegaByBSFormula(S0, sigma, r);
}
```

Units check

`Value` returns dollars. `Deriv` returns the price change per unit change in volatility. Their ratio is a volatility increment in Newton's update.

15 | Configure the implied-volatility solve – Problem

GUIDED CODE End-to-end integration — Fill the target, bracket, tolerance, and guess

```
Intermediary call(100.0, 0.10, 1.0, 100.0);

NonlinearSolver solver(
    /* target */, /* left */, /* right */,
    /* accuracy */, /* guess */);

double ivB = solver.SolveByBisect(&call);
double ivN = solver.SolveByNR(&call);
```

Market data

$C_{\text{quote}} = 12.56$, volatility bracket $[0.01, 1.00]$, tolerance 10^{-4}, initial guess 0.23.

15 | Configure the implied-volatility solve – Solution

```
NonlinearSolver solver(
    12.56, 0.01, 1.00, 0.0001, 0.23);

double ivB = solver.SolveByBisect(&call);
double ivN = solver.SolveByNR(&call);

std::cout << std::fixed << std::setprecision(4)
          << ivB << " " << ivN;
// 0.1784 0.1784
```

Interpretation

An annualized volatility near 17.84% makes the model price agree with the market quote to the requested accuracy.

Before trusting an implied volatility

1. Check the quote lies between prices at the bracket endpoints.
2. Check the reported volatility is positive and inside the bracket.
3. Reprice the option and inspect $|C(\hat{\sigma}) - C_{\text{quote}}|$.
4. Confirm vega is not too small near the Newton iterates.
5. Use a maximum iteration count and report failure clearly.

Do not validate only the step size

A small change in σ is useful, but the financial quantity we ultimately need to match is the option price.

Exit ticket

Explain

In one sentence each, state the role of:

- `Tgt`
- `LEnd`, `REnd`
- `Guess`
- `Acc`

Choose

Which method would you try first when:

- only function values are available?
- speed matters and vega is reliable?
- the initial guess is uncertain?

Next: put the solver behind a small, testable interface.

Reserve exercises

Use these only if the group is ready or as take-home practice.

R1 | Bond yield as a root – Problem

GUIDED CODE Challenge — Design `Value` and `Deriv`

For cash flows C_n at times T_n and face value F at maturity T,

$$P(y) = \sum_{n=1}^{N} C_n e^{-yT_n} + Fe^{-yT}.$$

Given a market price P_{mkt}, solve $P(y) = P_{\text{mkt}}$.

```
class BondYieldFunction : public Function {
    std::vector<double> cashFlow, time;
    double face, maturity;
public:
    double Value(double y);
    double Deriv(double y);
};
```

Question

What sign should `Deriv(y)` have for positive cash flows?

R1 | Bond yield as a root – Solution

```
double BondYieldFunction::Value(double y)
{
    double value = face * std::exp(-y * maturity);
    for (int n = 0; n < cashFlow.size(); ++n)
        value += cashFlow[n] * std::exp(-y * time[n]);
    return value;
}

double BondYieldFunction::Deriv(double y)
{
    double deriv = -maturity * face
                   * std::exp(-y * maturity);
    for (int n = 0; n < cashFlow.size(); ++n)
        deriv -= time[n] * cashFlow[n]
                 * std::exp(-y * time[n]);
    return deriv;
}
```

Sign: For positive future cash flows, price decreases as continuously compounded yield increases, so the derivative is negative.

R2 | One implied-volatility Newton step – Problem

GUIDED CODE Challenge Calculate before revealing

At the current guess $\sigma_0 = 0.2300$, suppose

$$C(\sigma_0) = 14.6100, \qquad \text{Vega}(\sigma_0) = 38.5000, \qquad C_{\text{quote}} = 12.5600.$$

Compute

$$\sigma_1 = \sigma_0 - \frac{C(\sigma_0) - C_{\text{quote}}}{\text{Vega}(\sigma_0)}.$$

Should the next volatility be higher or lower? Explain before calculating.

R2 | One implied-volatility Newton step – Solution

$$\sigma_1 = 0.2300 - \frac{14.6100 - 12.5600}{38.5000} = 0.17675.$$

Direction

The model price is too high and vega is positive. Reducing volatility reduces the call price, so the sign is economically sensible.

Numerical lesson

Newton can move close to the final 0.1784 answer in one step, but that speed depends on a useful derivative and a reasonable starting guess.

R3 | The diamond problem – Problem

GUIDED CODE Challenge — Count the base subobjects

```
class Person { public: int id; };
class Student  : public Person {};
class Employee : public Person {};
class GradAssistant : public Student,
                      public Employee {};

GradAssistant ga;
ga.id = 42;          // What happens?
```

Questions

How many `Person` subobjects are inside `ga`? Why is `ga.id` ambiguous?

R3 | The diamond problem – Solution

```
class Student : public virtual Person {};
class Employee : public virtual Person {};
class GradAssistant : public Student,
                      public Employee {};

GradAssistant ga;
ga.id = 42;          // one shared Person base
```

Meaning of `virtual` here

This use of `virtual` controls base-object sharing. It is related to inheritance layout, not virtual function dispatch.

R4 | Harden Newton's method – Find the bugs

GUIDED CODE Challenge

```
double SolveByNR(Function* f)
{
    double x = Guess;
    while (true) {
        double next = x - (f->Value(x) - Tgt)
                          / f->Deriv(x);
        if (next - x < Acc) return next;
        x = next;
    }
}
```

Identify at least four failure modes

R4 | Harden Newton's method – Find the bugs

GUIDED CODE Challenge

Identify at least four failure modes

```
double SolveByNR(Function* f)
{
    double x = Guess;
    while (true) {
        double next = x - (f->Value(x) - Tgt)
                          / f->Deriv(x);
        if (next - x < Acc) return next;
        x = next;
    }
}
```

Possible answers

Missing absolute value; no maximum iteration count; derivative may be zero or tiny; iterate may become non-finite; no residual check; no domain or bracket protection.

R5 | How many bisection steps?

After k bisection steps, the bracket width is

$$\frac{R - L}{2^k}.$$

Problem

Starting from the volatility bracket $[0.01, 1.00]$, how many steps guarantee a width below 10^{-4}?

R5 | How many bisection steps?

After k bisection steps, the bracket width is

$$\frac{R-L}{2^k}.$$

Problem

Starting from the volatility bracket $[0.01, 1.00]$, how many steps guarantee a width below 10^{-4}?

We need

$$k > \log_2\left(\frac{0.99}{10^{-4}}\right) \approx 13.27.$$

Therefore, **14 steps** guarantee the requested bracket width.

Nonlinear solvers – One-page cheat sheet

Bisection

Require a valid bracket. Evaluate the midpoint, keep the half that still brackets the target, and repeat. Slow, predictable, derivative-free.

Newton–Raphson

$x_{k+1} = x_k - [f(x_k) - c]/f'(x_k)$. Fast near the root; protect against a poor guess and a small derivative.

C++ designs

Function pointer: simple free functions.
Virtual interface: run-time polymorphism.
Class template: compile-time polymorphism.

Implied volatility

`Value`= Black–Scholes price; `Deriv`= vega; `Tgt`= market quote. Always reprice to verify the answer.